

Experiment 3

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Theory:

1. Overview of Blockchain

Blockchain is a **distributed, decentralized, and immutable digital ledger** used to record transactions securely across a peer-to-peer (P2P) network. Unlike traditional centralized databases, blockchain does not rely on a single authority. Instead, every participating node maintains a copy of the ledger.

Each block in a blockchain consists of:

- **Transaction data**
- **Timestamp**
Hash of the previous block
- **Current block hash**

The blocks are cryptographically linked using hashes, ensuring data integrity. Any modification in a block changes its hash and breaks the chain, making tampering computationally infeasible.

2. Cryptocurrency and Blockchain Simulation

A cryptocurrency is a **digital asset** that uses cryptographic techniques and blockchain technology to secure transactions and control the creation of new units. In this experiment, a **simple cryptocurrency (Hadcoin)** is implemented using Python to demonstrate real-world blockchain behavior.

The system simulates:

- Decentralized nodes
- Transaction handling
- Mining process
Consensus among peers

3. Mining Process

Mining is the process through which:

- Pending transactions are collected
 - A new block is created
 - A cryptographic puzzle (Proof-of-Work) is solved
 - The block is added to the blockchain
-
- In Proof-of-Work:
 - Miners repeatedly calculate hashes
 - A valid hash must satisfy a predefined difficulty condition (e.g., starting with leading zeros)
- Once found, the block is broadcast to all peers

The miner who successfully mines the block receives a **mining reward**, which is added as a transaction in the block.

4. Transactions in the Blockchain Network

A transaction represents the transfer of cryptocurrency between participants and contains:

- **Sender address**
- **Receiver address**
- **Amount**

Transactions are first added to a **temporary transaction pool (mempool)**. These pending transactions are later included in a mined block. After a block is successfully mined, the transactions are removed from the pool to prevent duplication.

5. Mempool (Memory Pool)

The mempool is a temporary storage area where **unconfirmed transactions** are kept before being mined into a block.

Role of mempool:

- Stores pending transactions
- Helps miners select transactions for the next block
- Prevents double spending
- Improves transaction organization and efficiency

In this experiment, the code is modified to ensure that transactions are **cleared from the mempool after being added to a block**.

6. Peer-to-Peer (P2P) Network

A P2P network allows nodes to communicate directly without a central server. In this lab:

- Three nodes run on ports **5001, 5002, and 5003**
- Each node maintains its own blockchain
- Nodes connect to peers using HTTP requests

This setup demonstrates decentralization, where no single node controls the network.

7. Challenges in P2P Networks

Some common challenges include:

- **Consensus management**: Ensuring all nodes agree on the same blockchain
- **Network latency**: Delay in block propagation
- **Forks**: Multiple versions of the blockchain
- **Security risks**: Sybil attacks, double spending
- **Scalability**: Handling large numbers of transactions

8. Consensus Mechanism – Longest Chain Rule

To maintain consistency, this experiment uses the **Longest Chain Rule**:

- If multiple chains exist, the longest valid chain is accepted
- Nodes compare their chain with peers
- The shorter chain is replaced automatically

This ensures synchronization and agreement across all nodes.

9. Public and Private Blockchain Implementation

- **Public Blockchain**: Multiple nodes participate and share the blockchain openly
- **Private Blockchain**: Controlled environment with selected nodes

This experiment demonstrates both concepts by allowing controlled node participation while maintaining decentralized behavior.

10. Tools and Libraries Used

Python : Used to implement blockchain logic, mining, hashing, and networking.

Flask: A lightweight web framework used to:

- Create REST APIs
- Handle blockchain requests such as mining, adding transactions, and chain replacement

1. Add Transactions - invoke `add_transactions()` as a POST request.

The screenshot shows the Postman interface for a POST request to `http://127.0.0.1:5001/add_transaction`. The request body is a JSON object:

```
1 {
2   "sender": "a",
3   "receiver": "b",
4   "amount": 53
5 }
6
```

The response is a 201 CREATED status with a message: "This transaction will be added to Block 2". The response body is:

```
1 {
2   "message": "This transaction will be added to Block 2"
3 }
```

The interface also shows a user profile for 'aryan' with email '2022.aryan.dangat@ves.ac.in' and options to view profile, settings, sign out, and switch accounts.

2. mining - `mine_block()`

The screenshot shows the Postman interface for a GET request to `http://127.0.0.1:5001/mine_block`. The response is a 200 OK status with a JSON array of four transactions:

```
18 {
19   "amount": 99,
20   "receiver": "b",
21   "sender": "a"
22 },
23 {
24   "amount": 49,
25   "receiver": "b",
26   "sender": "a"
27 },
28 {
29   "amount": 899,
30   "receiver": "b",
31   "sender": "a"
32 },
33 {
34   "amount": 1,
35   "receiver": "Richard",
36   "sender": "e44c976af0cd408baee3838a709e3016"
37 }
38 ]
39
```

The interface also shows a user profile for 'aryan' with email '2022.aryan.dangat@ves.ac.in' and options to view profile, settings, sign out, and switch accounts.

A Snipping Tool window is visible in the foreground, displaying the message: "Screenshot copied to clipboard. Automatically saved to screenshots folder. Markup and share".

3.fetch the chain - get_chain(),

http://127.0.0.1:5001/get_chain

GET http://127.0.0.1:5001/get_chain

Send

Docs Params Authorization Headers (6) Body Scripts Settings Cookies

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers (5) Test Results

200 OK · 4 ms · 533 B

{ } JSON Preview Visualize

```
1 {
2   "chain": [
3     {
4       "index": 1,
5       "previous_hash": "0",
6       "proof": 1,
7       "timestamp": "2026-01-27 11:09:26.829185",
8       "transactions": []
9     },
10    {
11      "index": 2,
12      "previous_hash": "6d017bf1d0543b614960a1afcdbe1e1bb6f80e0e048c18c818761faffe79dec0",
13      "proof": 533,
14      "timestamp": "2026-01-27 11:12:17.003830",
15    }
16  ]
17 }
```

http://127.0.0.1:5001/connect_node

POST http://127.0.0.1:5001/connect_node

Send

Docs Params Authorization Headers (8)

☐ none ☐ form-data ☐ x-www-form-urlencoded

```
1 {
2   "nodes":
3   [
4     "http://127.0.0.1:5002",
5     "http://127.0.0.1:5003"
6   ]
7 }
```

Body Cookies Headers (5) Test Results

201 CREATED · 4 ms · 325 B

{ } JSON Preview Visualize

```
1 {
2   "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following",
3   "nodes":
4   [
5     "127.0.0.1:5003",
6     "127.0.0.1:5002"
7   ]
8 }
```

arian
2022.aryan.dangat@ves.ac.in
View Profile
Settings
Sign Out
Switch Accounts
+ Add Account

Show desktop

Runner Start Proxy Cookies Vault Trash

search

ENG IN 09:43 30-01-2026

4. replace the longest chain - replace_chain()

http://127.0.0.1:5001/replace_chain

GET http://127.0.0.1:5001/replace_chain

200 OK · 3 ms · 1.2 KB

```
{
  "actual_chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": "2026-01-27 11:09:26.029105",
      "transactions": [
        {
          "amount": 1,
          "receiver": "Richard",
          "sender": "4208036bca004b56b2e456b4daff0059"
        }
      ]
    },
    {
      "index": 2,
      "previous_hash": "6d617bf1d8543b614960a1afcdbe1e1bb5f00e0e848c15c815761faffe79dec9",
      "proof": 533,
      "timestamp": "2026-01-27 11:12:17.083830",
      "transactions": [
        {
          "amount": 1,
          "receiver": "Richard",
          "sender": "4208036bca004b56b2e456b4daff0059"
        }
      ]
    },
    {
      "index": 3,
      "previous_hash": "1af95c07c0d568f49621eb2fd8759ff3c729423d818dae19a05b2083fd",
      "proof": 45293,
      "timestamp": "2026-01-27 11:19:00.005438",
      "transactions": [
        {
          "amount": 1,
          "receiver": "Richard",
          "sender": "4208036bca004b56b2e456b4daff0059"
        }
      ]
    }
  ]
}
```

```

"index": 4,
"previous_hash": "ed3cc6f96be22e405a6ed26538e3c0a71a79779d602eff6e7e713be9ec690104",
"proof": 21391,
"timestamp": "2026-01-27 11:26:58.840324",
"transactions": [
    {
        "amount": 1,
        "receiver": "Richard",
        "sender": "4208036bca004b56b2e456b4daff0059"
    },
    {
        "amount": 1000,
        "receiver": "pooja",
        "sender": "4208036bca004b56b2e456b4daff0059"
    },
    {
        "amount": 1,
        "receiver": "Richard",

```

```

55         "sender": "4208036bca004b56b2e456b4daff0059"
56     },
57     ]
58 },
59 ],
60 "message": "All good. The chain is the largest one."
61 }

```

Conclusion

This experiment demonstrates the practical implementation of a cryptocurrency using Python and blockchain principles. By simulating a P2P network with multiple nodes, mining, transactions, and consensus mechanisms, the experiment provides hands-on understanding of how real-world blockchain systems operate. It highlights the importance of decentralization, security, and consensus in distributed ledger technologies.